

Young Innovators · Class 10

# Wireless Doorbell with Visitor Counter

Two Arduinos talking by radio!

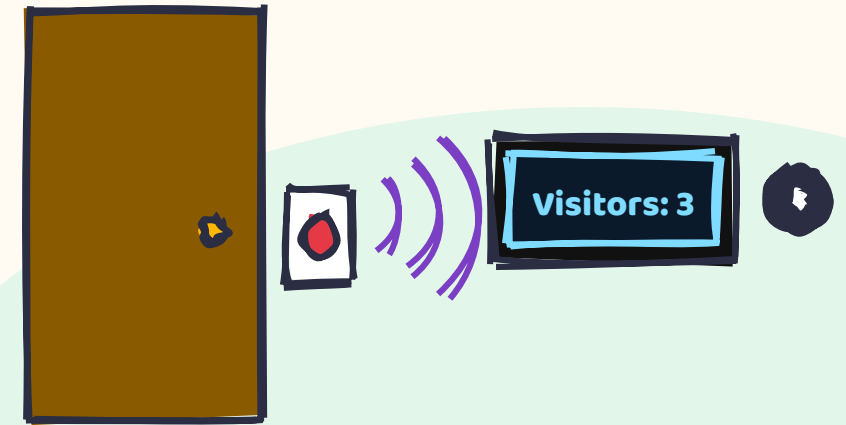
NRF24L01 radio

Addresses

Two-device system

Did you know?

Our radios use 2.4 GHz, the same band as Wi-Fi, Bluetooth and even microwave ovens.





Let's think...

# How does the bell know?

## The wire problem

An old doorbell needs a **wire** from the gate to inside the house. Drilling through walls is messy and expensive. Long wires break. Wireless doorbells need **no wire at all**.



A very long wire?

Shout louder?

Send a radio message!

**Today:** pairs of groups build a **transmitter** for the door and a **receiver** that rings and counts visitors.





## Learning aims

# Today I will learn to...



**1**  Explain how **radio waves** carry messages

**2**  Wire an **NRF24L01** safely to an Uno (3.3 V!)

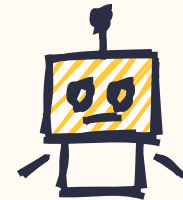
**3**  Use matching **addresses** so the right devices talk

**4**  Program a **transmitter** and a **receiver** that rings and counts

**5**  Test the **range** through walls and explain the results

**Keywords**

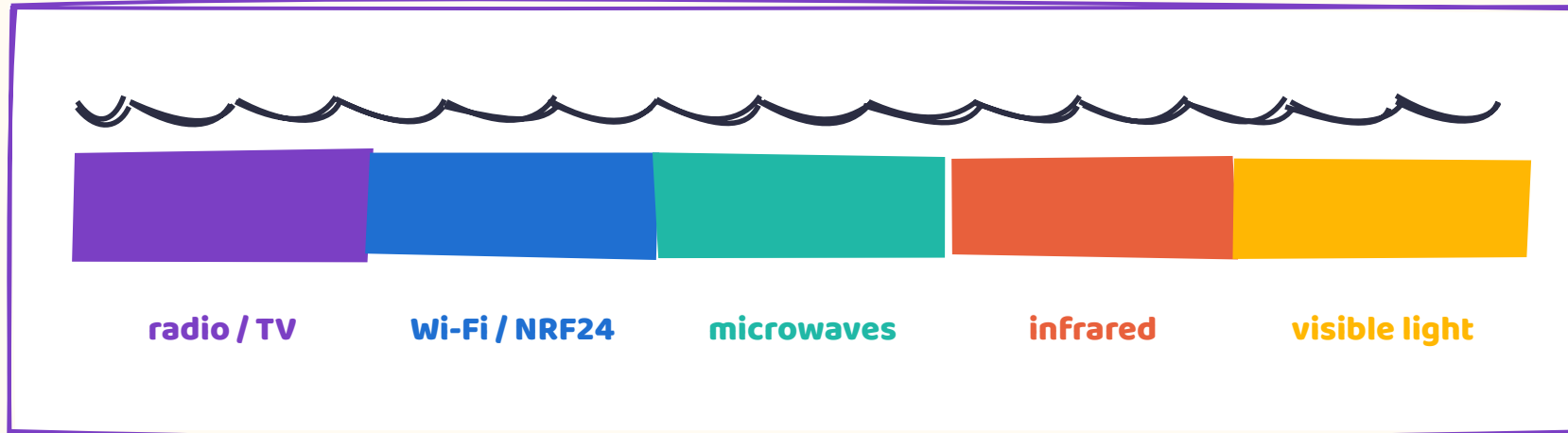
radio · 2.4 GHz · transmitter ·  
receiver · address · pipe · ACK





## The science

# Radio waves: light we cannot see



## 2.4 GHz

Our radio wave wiggles **2.4 billion times every second**. It travels at the speed of light, so the message reaches the receiver almost instantly.

## Walls and water

Radio passes through wood and bricks, but gets weaker. **Metal** and **water** (even people!) block it more.

## Crowded band

Many devices share 2.4 GHz. Good radios use addresses and channels to avoid mix-ups.





Real world

# Short-range radio is everywhere



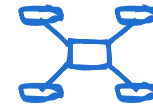
## Wireless bells

Button outside, chime inside:  
exactly our project



## Car keys

Press the fob, a coded radio  
message unlocks the car



## Drone remotes

Send joystick moves many  
times a second



## Walkie-talkies

Send voices on a shared radio  
channel



Your car key and your neighbour's look the same. Why doesn't yours open their car?

### Hint

Each key sends a different CODE. Our  
radios use ADDRESSES.





Meet the radio

# The NRF24L01 module



## 3.3 V only!

Connecting VCC to 5V can **destroy** the module. Double-check before plugging in USB.

## SPI

The module talks to the Uno through **SPI**, a fast 4-wire connection (SCK, MOSI, MISO, CSN). Pins 11, 12, 13 are fixed for SPI on the Uno.



|             |                |
|-------------|----------------|
| VCC         | 3.3V (NOT 5V!) |
| GND         | GND            |
| CE / CSN    | 9 / 10         |
| SCK         | 13             |
| MOSI / MISO | 11 / 12        |
| IRQ         | not used       |

## Add a capacitor

A **10  $\mu$ F** capacitor across VCC and GND (long leg to VCC) smooths the power. Without it, messages often get lost.

## Pin diagram

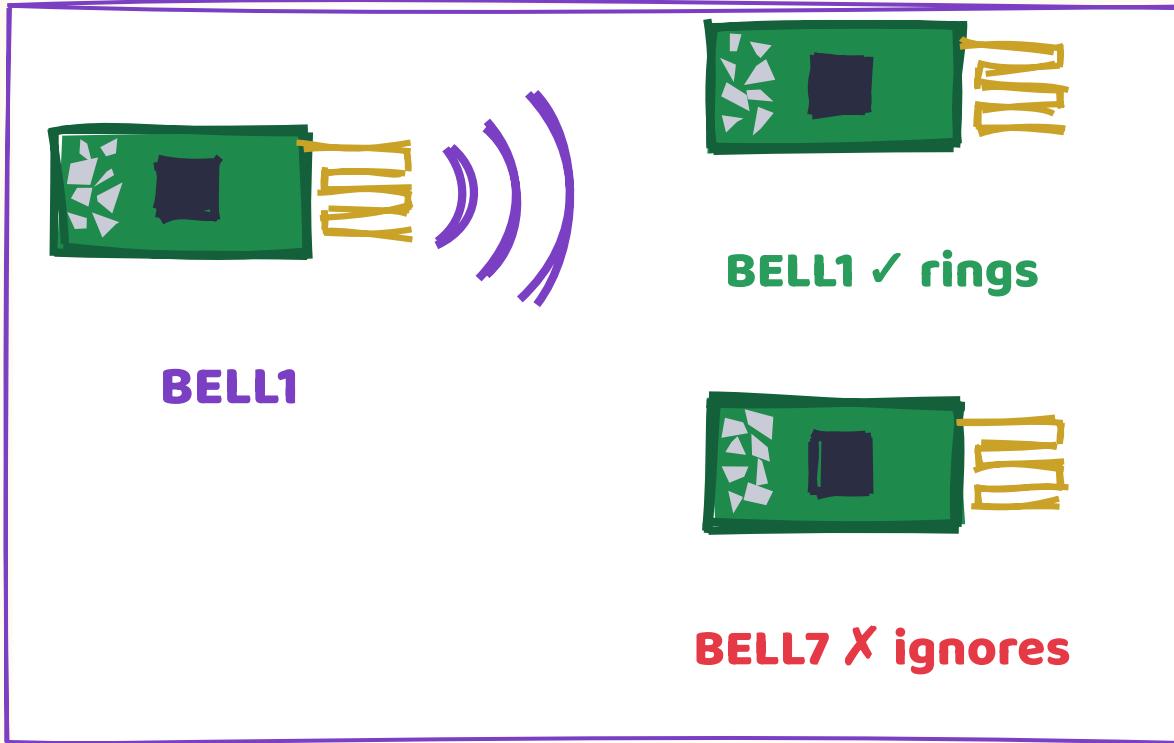
The pins are not labelled on most boards! Use the diagram your teacher gives you, looking from the top.





Big idea

# Addresses: a name on the envelope



## Matching address

The transmitter writes to address "**BELL1**". Only a receiver listening on "**BELL1**" accepts the message. Up to 5 letters.

## Classroom rule!

10 groups in one room = 10 doorbells. Every pair must pick its **own** address (BELL1, BELL2, ...), or everyone's chime rings!





How they talk

# Send, then wait for a 'got it!'



## Who does what?

**Door unit:** button + transmitter.  
**Inside unit:** receiver + buzzer + OLED counter.



## Automatic acknowledgement

The NRF24 sends back a tiny **ACK** message by itself. `radio.write()` returns **true** if the ACK arrived. If not, it tries again a few times.

## Use the ACK!

Light an LED on the door unit when the ACK arrives, so the visitor knows the bell rang.



## Class game

# Radio post office

### How to play

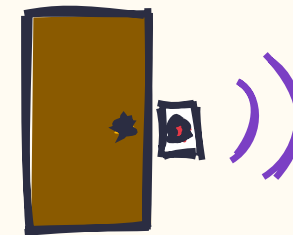
1. Each pair writes an **address** on a card (BELL1, BELL2...).
2. Senders write a message and the address on a paper slip.
3. The "radio" (teacher) shouts every message out loud to the whole room.
4. Only the pair with the matching address may reply "ACK!"
5. Then: two senders shout at the same time. What happens?

### Then discuss

Why did shouting at the same time cause problems? How could radios take turns?

### Big idea

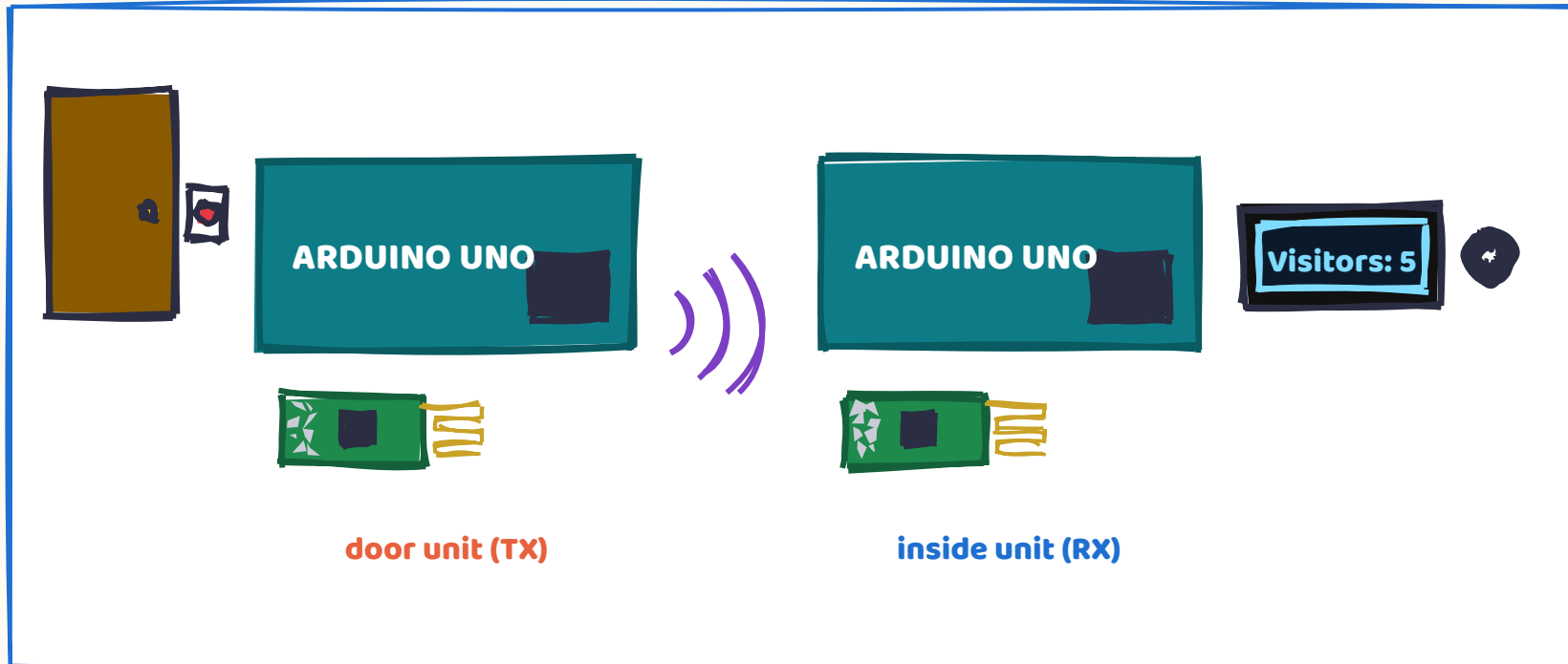
Everyone hears every radio message. Addresses decide who listens.





The plan

# Two Arduinos, one system



## Pair up!

Two groups join together. One builds the **door unit**, one builds the **inside unit**. You must agree the address and the message.



## System thinking

If one half fails, the whole system fails.  
Test each half on its own first!





Build it

# Our doorbell kit (per pair of groups)



**2 × Arduino Uno**  
TX and RX



**2 × NRF24L01**  
The radios



**2 × 10  $\mu$ F capacitor**  
Power smoothing



**Push button + 10k**  
The door button



**OLED 0.96 in**  
Visitor counter



**Buzzer**  
The chime

Also: 2 breadboards, jumper wires (M-F for the radios), 2 USB cables, 2 laptops, and the RF24 library by TMRh20.

**Power for range tests**

Use a power bank on the door unit so you can walk around.





Build it

# Wiring both units



| NRF24L01 (both units) | Uno                |
|-----------------------|--------------------|
| VCC                   | 3.3V               |
| GND                   | GND                |
| CE / CSN              | 9 / 10             |
| SCK / MOSI / MISO     | 13 / 11 / 12       |
| 10 $\mu$ F capacitor  | across VCC and GND |

| Door unit |             |
|-----------|-------------|
| Button    | 5V → pin 2  |
| 10k       | pin 2 → GND |

| Inside unit    |         |
|----------------|---------|
| OLED SDA / SCL | A4 / A5 |
| Buzzer +       | pin 8   |

## Before you plug in USB

Ask a partner to check: VCC on **3.3V**? Capacitor the right way round?  
Then the teacher checks too.



Code it: door unit

# The transmitter

```
#include <SPI.h>
#include <RF24.h>
RF24 radio(9, 10);           // CE, CSN
const byte address[6] = "BELL1"; // your pair's address
const int BUTTON = 2;

void setup() {
  pinMode(BUTTON, INPUT);
  radio.begin();
  radio.openWritingPipe(address);
  radio.setPALevel(RF24_PA_LOW);
  radio.stopListening();      // I am the sender
}

void loop() {
  if (digitalRead(BUTTON) == HIGH) {
    const char msg[] = "RING";
    bool ok = radio.write(&msg, sizeof(msg));
    delay(1000);              // one ring per press
  }
}
```

## Key lines

**openWritingPipe:** who I send to.

**stopListening:** I am a sender.

**write:** send the bytes, returns true if ACK came back.

## Why delay(1000)?

Holding the button for half a second would send many RINGs. The delay means one press = one visitor.



Code it: inside unit

# The receiver and counter

```
RF24 radio(9, 10);
const byte address[6] = "BELL1"; // must match!
int visitors = 0;

void setup() {
  radio.begin();
  radio.openReadingPipe(0, address);
  radio.setPALevel(RF24_PA_LOW);
  radio.startListening();          // I am the receiver
  // + OLED setup
}

void loop() {
  if (radio.available()) {
    char msg[32] = "";
    radio.read(&msg, sizeof(msg));
    if (strcmp(msg, "RING") == 0) {
      visitors++;
      tone(8, 880, 300); delay(350);
      tone(8, 660, 400);          // ding-dong!
      // show visitors on the OLED
    }
  }
}
```

## Key lines

**openReadingPipe:** listen on my address.

**available:** has a message arrived?

**strcmp:** is it exactly "RING"? (0 means equal)

## Why check the message?

Later you can send other messages ("KNOCK", "PARCEL") and react to each one differently.





## Experiment

# How far can the bell reach?



| Door unit position       | PA_LOW rings? | PA_MAX rings? |
|--------------------------|---------------|---------------|
| Same table (1 m)         | yes / no      | yes / no      |
| Across the room (8 m)    | yes / no      | yes / no      |
| Through 1 wall           | yes / no      | yes / no      |
| Down the corridor (20 m) | yes / no      | yes / no      |
| Through 2 walls          | yes / no      | yes / no      |

### Fair test

Keep the inside unit in one place.  
Press 5 times at each spot. Ring 5 out of 5 = reliable.

### Explain your results

Where did it stop working? Was it distance or the walls? What else was in the way: metal doors, people, water pipes?

### PA levels

PA\_MIN, LOW, HIGH, MAX: more power = more range, but more battery used.





Test like an engineer

# Debug like a radio engineer



| Problem                    | Try this                                       |
|----------------------------|------------------------------------------------|
| Nothing ever arrives       | VCC on 3.3V? Capacitor fitted?                 |
| Works sometimes only       | add/replace the capacitor, shorter wires       |
| Another group's bell rings | addresses are the same: change yours           |
| Upload error               | check the RF24 library is installed            |
| Rings many times per press | add the delay, or detect the button press edge |

## Debug checklist

Print **ok** from `radio.write()` to the Serial monitor. `true` = the ACK came back.  
Test both units on one table first.



Swap ONE part at a time with a working pair (radio, then wires, then Arduino) to find the faulty part.



Level up!

# Doorbell challenges

## Choose your challenge



**Visit times:** send millis() and show "minutes since switch-on" for the last visitor.



**Two doors:** add a back-door button that sends a different message and a different tune.



**Do not disturb:** a button on the inside unit that silences the bell but still counts.



**Reply:** make the inside unit send "COMING" back, and light an LED at the door.

## Think like an engineer

Anyone with an NRF24 who knows your address could ring your bell. How could you make it more secure?

## Boss level

Send a secret code number with each RING, and only ring if the code is right.



Show what you know

# What did we learn?

- ☒ Radio waves carry messages without wires
- ☒ The NRF24L01 needs 3.3 V and a capacitor
- ☒ Matching addresses decide who listens
- ☒ The ACK tells the sender the message arrived
- ☒ Walls, metal and distance reduce range



**Exit ticket:** why must every pair in the class use a different address?



## At home

Find 3 wireless gadgets at home. Do they use Wi-Fi, Bluetooth or another radio?



**Next class:** a Wi-Fi Automatic Plant Watering System!

